

SKILL-04

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

CODE:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100

/* Token types */
typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_BACKGROUND
} TokenType;

/* Token structure */
typedef struct {
    TokenType type;
    char *value;
} Token;

/* Parse tree node */
typedef struct CommandNode {
    char **arguments;
    int argumentCount;
    char *inputFile;
    char *outputFile;
    int append;
    int background;
    struct CommandNode *next;
} CommandNode;

/* Return token type as text */
const char *tokenTypeName(TokenType type)
{
    switch (type) {
        case TOKEN_WORD:
```

```

        return "WORD";

    case TOKEN_PIPE:
        return "PIPE";

    case TOKEN_INPUT:
        return "INPUT";

    case TOKEN_OUTPUT:
        return "OUTPUT";

    case TOKEN_APPEND:
        return "APPEND";

    case TOKEN_BACKGROUND:
        return "BACKGROUND";

    default:
        return "UNKNOWN";
    }
}

/* Create a copy of a string */
char *copyString(const char *source)
{
    char *result;

    result = malloc(strlen(source) + 1);

    if (result == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    strcpy(result, source);

    return result;
}

/* Add token to token array */
void addToken(Token tokens[], int *count, TokenType type,
               const char *value)
{
    if (*count >= MAX_TOKENS) {
        printf("Error: Too many tokens.\n");
        return;
    }

    tokens[*count].type = type;
    tokens[*count].value = copyString(value);
}

```

```

    (*count)++;
}

/* Tokenize input */
int tokenize(const char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0') {

        /* Ignore whitespace */
        if (isspace((unsigned char)input[i])) {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|') {
            addToken(tokens, &count, TOKEN_PIPE, "|");
            i++;
            continue;
        }

        /* Input redirection */
        if (input[i] == '<') {
            addToken(tokens, &count, TOKEN_INPUT, "<");
            i++;
            continue;
        }

        /* Output redirection or append */
        if (input[i] == '>') {
            if (input[i + 1] == '>') {
                addToken(tokens, &count, TOKEN_APPEND, ">>");
                i += 2;
            } else {
                addToken(tokens, &count, TOKEN_OUTPUT, ">");
                i++;
            }
        }

        continue;
    }

    /* Background */
    if (input[i] == '&') {
        addToken(tokens, &count, TOKEN_BACKGROUND, "&");
        i++;
    }
}

```

```

        continue;
    }

    /* Normal word */
    {
        char buffer[256];
        int length = 0;

        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]) &&
               input[i] != '|' &&
               input[i] != '<' &&
               input[i] != '>' &&
               input[i] != '&') {

            if (length < 255) {
                buffer[length++] = input[i];
            }

            i++;
        }

        buffer[length] = '\0';

        if (length > 0) {
            addToken(tokens, &count,
                    TOKEN_WORD, buffer);
        }
    }
}

return count;
}

/* Display tokens */
void printTokens(Token tokens[], int count)
{
    int i;

    printf("\n----- TOKENS ----- \n");

    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }

    for (i = 0; i < count; i++) {
        printf("Token %d: %-12s Value: %s\n",
               i + 1,
               tokenTypeName(tokens[i].type),

```

```

        tokens[i].value);
    }
}

/* Free tokens */
void freeTokens(Token tokens[], int count)
{
    int i;

    for (i = 0; i < count; i++) {
        free(tokens[i].value);
        tokens[i].value = NULL;
    }
}

/* Create command node */
CommandNode *createCommandNode(void)
{
    CommandNode *node;

    node = malloc(sizeof(CommandNode));

    if (node == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    node->arguments = NULL;
    node->argumentCount = 0;
    node->inputFile = NULL;
    node->outputFile = NULL;
    node->append = 0;
    node->background = 0;
    node->next = NULL;

    return node;
}

/* Add argument to command */
void addArgument(CommandNode *node, const char *argument)
{
    char **temp;

    temp = realloc(node->arguments,
        sizeof(char *) *
        (node->argumentCount + 1));

    if (temp == NULL) {
        perror("realloc");
        exit(EXIT_FAILURE);
    }

```

```

    }

    node->arguments = temp;

    node->arguments[node->argumentCount] =
        copyString(argument);

    node->argumentCount++;
}

/* Validate syntax */
int validateSyntax(Token tokens[], int count)
{
    int i;

    if (count == 0) {
        printf("Syntax Error: Empty command.\n");
        return 0;
    }

    /* Command cannot start with pipe */
    if (tokens[0].type == TOKEN_PIPE) {
        printf("Syntax Error: Command cannot start with '|'.\n");
        return 0;
    }

    /* Command cannot end with pipe */
    if (tokens[count - 1].type == TOKEN_PIPE) {
        printf("Syntax Error: Command cannot end with '|'.\n");
        return 0;
    }

    for (i = 0; i < count; i++) {

        /* Two pipes together */
        if (tokens[i].type == TOKEN_PIPE &&
            i + 1 < count &&
            tokens[i + 1].type == TOKEN_PIPE) {

            printf("Syntax Error: Consecutive pipes.\n");
            return 0;
        }

        /* Redirection needs a filename */
        if (tokens[i].type == TOKEN_INPUT ||
            tokens[i].type == TOKEN_OUTPUT ||
            tokens[i].type == TOKEN_APPEND) {

            if (i + 1 >= count ||
                tokens[i + 1].type != TOKEN_WORD) {

```

```

        printf("Syntax Error: Redirection requires a filename.\n");
        return 0;
    }
}

/* Background should be last */
if (tokens[i].type == TOKEN_BACKGROUND &&
    i != count - 1) {

    printf("Syntax Error: '&' must be at the end.\n");
    return 0;
}

return 1;
}

/* Build parse tree */
CommandNode *parseTokens(Token tokens[], int count)
{
    CommandNode *head = NULL;
    CommandNode *current = NULL;

    int i;

    for (i = 0; i < count; i++) {

        /* New command after pipe */
        if (current == NULL) {

            CommandNode *newNode = createCommandNode();

            if (head == NULL) {
                head = newNode;
            } else {
                current->next = newNode;
            }

            current = newNode;
        }

        /* Word */
        if (tokens[i].type == TOKEN_WORD) {

            /* Redirection filename */
            if (i > 0 &&
                (tokens[i - 1].type == TOKEN_INPUT ||
                 tokens[i - 1].type == TOKEN_OUTPUT ||
                 tokens[i - 1].type == TOKEN_APPEND)) {

```

```

        if (tokens[i - 1].type == TOKEN_INPUT) {
            current->inputFile =
                copyString(tokens[i].value);
        } else {
            current->outputFile =
                copyString(tokens[i].value);

            if (tokens[i - 1].type == TOKEN_APPEND) {
                current->append = 1;
            }
        }

    } else {
        addArgument(current, tokens[i].value);
    }
}

/* Pipe */
else if (tokens[i].type == TOKEN_PIPE) {
    current = NULL;
}

/* Background */
else if (tokens[i].type == TOKEN_BACKGROUND) {
    current->background = 1;
}
}

return head;
}

/* Print parse tree */
void printParseTree(CommandNode *head)
{
    CommandNode *current = head;
    int commandNumber = 1;
    int i;

    printf("\n----- PARSE TREE / EXECUTION STRUCTURE ----- \n");

    while (current != NULL) {

        printf("\nCommand %d\n", commandNumber);

        printf("Arguments: ");

        for (i = 0; i < current->argumentCount; i++) {
            printf("[%s] ", current->arguments[i]);
        }
    }
}

```

```

    if (current->argumentCount == 0) {
        printf("None");
    }

    printf("\n");

    if (current->inputFile != NULL) {
        printf("Input file : %s\n",
            current->inputFile);
    } else {
        printf("Input file : None\n");
    }

    if (current->outputFile != NULL) {
        printf("Output file: %s",
            current->outputFile);

        if (current->append) {
            printf(" (Append)");
        }

        printf("\n");
    } else {
        printf("Output file: None\n");
    }

    printf("Background : %s\n",
        current->background ? "Yes" : "No");

    current = current->next;
    commandNumber++;
}

/* Free parse tree */
void freeParseTree(CommandNode *head)
{
    CommandNode *current = head;
    CommandNode *next;
    int i;

    while (current != NULL) {

        next = current->next;

        for (i = 0; i < current->argumentCount; i++) {
            free(current->arguments[i]);
        }
    }
}

```

```

        free(current->arguments);
        free(current->inputFile);
        free(current->outputFile);
        free(current);

        current = next;
    }
}

int main(void)
{
    char input[1024];

    Token tokens[MAX_TOKENS];

    int tokenCount;

    CommandNode *parseTree;

    printf("=====\n");
    printf("    TOKENIZER AND PARSER\n");
    printf("=====\n");

    printf("Enter a shell-like command.\n");
    printf("Examples:\n");
    printf("  ls -l -a\n");
    printf("  cat file.txt | grep hello\n");
    printf("  sort < input.txt > output.txt\n");
    printf("  echo hello >> output.txt &\n");
    printf("Type 'exit' to quit.\n");
    printf("-----\n");

    while (1) {

        printf("\nparser> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        /* Handle empty command */
        if (strlen(input) == 0) {
            printf("Empty command. Nothing to parse.\n");
            continue;
        }
    }
}

```

```

/* Exit */
if (strcmp(input, "exit") == 0) {
    break;
}

/* Clear token array */
memset(tokens, 0, sizeof(tokens));

/* Tokenize */
tokenCount = tokenize(input, tokens);

printTokens(tokens, tokenCount);

/* Validate syntax */
printf("\n----- SYNTAX VALIDATION ----- \n");

if (!validateSyntax(tokens, tokenCount)) {
    printf("Result: INVALID SYNTAX\n");

    freeTokens(tokens, tokenCount);
    continue;
}

printf("Result: VALID SYNTAX\n");

/* Generate parse tree */
parseTree = parseTokens(tokens, tokenCount);

if (parseTree == NULL) {
    printf("Error: Could not generate parse tree.\n");
} else {
    printParseTree(parseTree);
    freeParseTree(parseTree);
}

/* Free token memory */
freeTokens(tokens, tokenCount);
}

printf("\nParser terminated successfully.\n");

return 0;
}

```

OUTPUT:-

```
venu@LAPTOP-KHLR30TF: ~/ $ cd ~/os-lab/skill
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ nano skill-04.c
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ gcc -Wall -Wextra -g skill-04.c -o skill-04
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ ./skill-04
=====
TOKENIZER AND PARSER
=====
Enter a shell-like command.
Examples:
  ls -l -a
  cat file.txt | grep hello
  sort < input.txt > output.txt
  echo hello >> output.txt &
Type 'exit' to quit.
-----

parser> ls -l -a

----- TOKENS -----
Token 1: WORD      Value: ls
Token 2: WORD      Value: -l
Token 3: WORD      Value: -a

----- SYNTAX VALIDATION -----
Result: VALID SYNTAX

----- PARSE TREE / EXECUTION STRUCTURE -----

Command 1
Arguments: [ls] [-l] [-a]
Input file : None
Output file: None
Background : No

parser> cat file.txt | grep hello

----- TOKENS -----
Token 1: WORD      Value: cat
Token 2: WORD      Value: file.txt
Token 3: PIPE       Value: |
Token 4: WORD      Value: grep
```

```
----- SYNTAX VALIDATION -----
Result: VALID SYNTAX
Segmentation fault ./skill-04
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ sort < input.txt > output.txt
-bash: input.txt: No such file or directory
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ ./skill-04
-bash: ./skill-04: No such file or directory
venu@LAPTOP-KHLR30TF: ~/os-lab/skill$ ./skill-04
=====
TOKENIZER AND PARSER
=====
Enter a shell-like command.
Examples:
  ls -l -a
  cat file.txt | grep hello
  sort < input.txt > output.txt
  echo hello >> output.txt &
Type 'exit' to quit.
-----

parser> sort < input.txt > output.txt

----- TOKENS -----
Token 1: WORD      Value: sort
Token 2: INPUT     Value: <
Token 3: WORD      Value: input.txt
Token 4: OUTPUT    Value: >
Token 5: WORD      Value: output.txt

----- SYNTAX VALIDATION -----
Result: VALID SYNTAX

----- PARSE TREE / EXECUTION STRUCTURE -----

Command 1
Arguments: [sort]
Input file : input.txt
Output file: output.txt
Background : No
```

```
venu@LAPTOP-KHLR30TF: ~/  x  +  v

parser> echo hello >> output.txt &

----- TOKENS -----
Token 1: WORD      Value: echo
Token 2: WORD      Value: hello
Token 3: APPEND    Value: >>
Token 4: WORD      Value: output.txt
Token 5: BACKGROUND Value: &

----- SYNTAX VALIDATION -----
Result: VALID SYNTAX

----- PARSE TREE / EXECUTION STRUCTURE -----

Command 1
Arguments: [echo] [hello]
Input file : None
Output file: output.txt (Append)
Background : Yes

parser> | hello

----- TOKENS -----
Token 1: PIPE      Value: |
Token 2: WORD      Value: hello

----- SYNTAX VALIDATION -----
Syntax Error: Command cannot start with '|'.
Result: INVALID SYNTAX

parser> hello |

----- TOKENS -----
Token 1: WORD      Value: hello
Token 2: PIPE      Value: |

----- SYNTAX VALIDATION -----
Syntax Error: Command cannot end with '|'.
Result: INVALID SYNTAX
```

```
Result: INVALID SYNTAX

parser> hello || world

----- TOKENS -----
Token 1: WORD      Value: hello
Token 2: PIPE      Value: |
Token 3: PIPE      Value: |
Token 4: WORD      Value: world

----- SYNTAX VALIDATION -----
Syntax Error: Consecutive pipes.
Result: INVALID SYNTAX

parser> cat <

----- TOKENS -----
Token 1: WORD      Value: cat
Token 2: INPUT     Value: <

----- SYNTAX VALIDATION -----
Syntax Error: Redirection requires a filename.
Result: INVALID SYNTAX

parser> valgrind --leak-check=full --show-leak-kinds=all ./skill-04

----- TOKENS -----
Token 1: WORD      Value: valgrind
Token 2: WORD      Value: --leak-check=full
Token 3: WORD      Value: --show-leak-kinds=all
Token 4: WORD      Value: ./skill-04

----- SYNTAX VALIDATION -----
Result: VALID SYNTAX

----- PARSE TREE / EXECUTION STRUCTURE -----

Command 1
Arguments: [valgrind] [--leak-check=full] [--show-leak-kinds=all] [./skill-04]
Input file : None
Output file: None
```

```
venu@LAPTOP-KHLR30TF: ~/ / + v
Token 1: WORD      Value: cat
Token 2: INPUT     Value: <

----- SYNTAX VALIDATION -----
Syntax Error: Redirection requires a filename.
Result: INVALID SYNTAX

parser> valgrind --leak-check=full --show-leak-kinds=all ./skill-04

----- TOKENS -----
Token 1: WORD      Value: valgrind
Token 2: WORD      Value: --leak-check=full
Token 3: WORD      Value: --show-leak-kinds=all
Token 4: WORD      Value: ./skill-04

----- SYNTAX VALIDATION -----
Result: VALID SYNTAX

----- PARSE TREE / EXECUTION STRUCTURE -----

Command 1
Arguments: [valgrind] [--leak-check=full] [--show-leak-kinds=all] [./skill-04]
Input file : None
Output file: None
Background : No

parser> exit

Parser terminated successfully.
venu@LAPTOP-KHLR30TF:~/os-lab/skill$
```